

Unit III: Packages, Utility Classes & Collections

IT405PC JAVA PROGRAMMING

Shafakhatullah Khan Mohammed

Department of Information Technology



ACE Engineering College

Packages | Utility Classes | Collections

Table of Contents

Packages

- Defining a Package
- CLASSPATH
- Access Specifiers
- Importing Packages

Utility Classes

- StringTokenizer
- BitSet
- Date and Calendar
- Random
- Formatter and Scanner

Collections

- Collections Overview
- Collection Interfaces
- Implementation Classes
- Sorting
- Comparable and Comparator

Definition

A **Package** is a **namespace** that organizes related **classes** and **interfaces** together, similar to folders in a file system. It prevents naming conflicts and controls access between classes.

Why Use Packages?

Avoids **Name Collision**

Provides **Access Protection**

Improves **Code Organization**

Promotes **Reusability**

Types of Packages

Built-in: `java.lang`, `java.util`, `java.io`

User-defined: Created by programmer

Defining a Package

Syntax

```
package packageName;  
package com.company.project;
```

Rules

- Must be the **first line** of code
- Only **one** package statement per file
- Use **lowercase** (convention)
- Sub-packages use **dot (.)** separator

Compilation

Syntax

```
javac -d . Hello.java
```

Hello.java

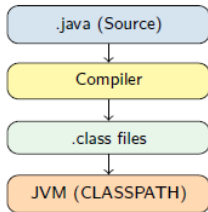
```
package IT405PC;  
  
public class Hello{  
    public void display(){  
        System.out.println("Hello from mypack!");  
    }  
}
```

CLASSPATH

Definition

CLASSPATH is an environment variable that tells the **JVM** and **Java compiler** where to find the compiled **.class** files and packages.

How it Works



Setting CLASSPATH

Windows

```
set CLASSPATH=C:\myproject;.
```

Linux

```
export CLASSPATH=/myproject:.
```

Flag Method

```
javac -classpath C:\proj Hello.java  
java -classpath C:\proj Hello
```

Note

"."(dot) = current directory

Access Specifiers

Definition

Access Specifiers define the **visibility** of classes, methods, and variables in Java.

Modifier	Same Class	Same Package	Sub-Class	Other Package
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Note

Memory Aid: private < default < protected < public

Access Specifiers – Example

AccessDemo.java

```
-----  
public class AccessDemo{  
    private int a = 10; // same class  
    int b = 20; // same pkg  
    protected int c = 30; // subclass+pkg  
    public int d = 40; //everywhere  
    private void show(){  
        System.out.println("a="+a);  
        System.out.println("b="+b);  
        System.out.println("c="+c);  
        System.out.println("d="+d);  
    }  
    public static void main(String[] a){  
        AccessDemo obj = new AccessDemo();  
        obj.show();  
    }  
}
```

Output

```
a=10  
b=20  
c=30  
d=40
```

Explanation

- a only within AccessDemo
- b within same package
- c subclasses + same package
- d accessible anywhere

Importing Packages

Definition

The **import** statement lets you use classes from other packages without their **fully qualified name**.

Syntax

```
import packageName.ClassName;  
import packageName.*;
```

Three Ways to Use a Class

1 Fully Qualified Name:

```
java.util.Scanner sc = new java.util.Scanner(System.in);
```

2 Import Specific Class:

```
import java.util.Scanner;
```

3 Import All (Wildcard):

```
import java.util.*;
```


Import User-Defined Package

Step 1: Create Package

```
mypack/Calc.java
-----
public class Calc{
    public int add(int a,int b){ return a + b; }
    public int sub(int a,int b){ return a - b; }
}
```

Step 2: Use Package

```
Main.java
-----
public class Main{
    public static void main( String[] a){
        Calc c = new Calc();
        System.out.println( c.add(10,5));
        System.out.println( c.sub(10,5));
    }
}
```

Output

```
15
5
```

StringTokenizer – Definition

Definition

StringTokenizer breaks a **String** into parts called **tokens** based on a **delimiter**. Default delimiter is **whitespace**.

Syntax

```
StringTokenizer st = new StringTokenizer(str);  
StringTokenizer st = new StringTokenizer(str, ",");
```

Important Methods

<code>hasMoreTokens()</code>	true if tokens remain
<code>nextToken()</code>	returns next token
<code>countTokens()</code>	returns remaining count
<code>hasMoreElements()</code>	same as <code>hasMoreTokens()</code>
<code>nextElement()</code>	same as <code>nextToken()</code>

StringTokenizer – Example

STDemo.java

```
1 import java.util.StringTokenizer;
2 class STDemo {
3     public static void main(String[] a){
4         // Default delimiter (space)
5         String s1 = "Hello World Java";
6         StringTokenizer st1 = new StringTokenizer(s1);
7         System.out.println("Total: " + st1.countTokens());
8         while(st1.hasMoreTokens()){
9             System.out.println(st1.nextToken());
10        }
11        // Custom delimiter (comma)
12        String s2 = "Apple,Mango,Banana";
13        StringTokenizer st2 = new StringTokenizer(s2, ",");
14        while(st2.hasMoreTokens()){
15            System.out.println(st2.nextToken());
16        }
17    }
18 }
```

Output

```
Total: 3
Hello
World
Java
Apple
Mango
Banana
```

Explanation

countTokens() returns 3

hasMoreTokens() checks remaining

nextToken() gets each token

Comma delimiter in Example 2

BitSet – Definition

Definition

BitSet creates a **dynamic array of bits** (0s and 1s). Each bit is **SET** (true) or **CLEAR** (false). Memory-efficient for booleans.

Syntax

```
BitSet bs = new BitSet();  
BitSet bs = new BitSet(16);
```

Important Methods

set(index)	set bit to true
clear(index)	set bit to false
get(index)	get bit value
flip(index)	toggle bit
and(BitSet)	logical AND
or(BitSet)	logical OR
xor(BitSet)	logical XOR
size()	size of BitSet

BitSetDemo.java

```
1- import java.util.BitSet;
2- public class BitSetDemo{
3-     public static void main(String[] a){
4-         BitSet bs1 = new BitSet(8); BitSet bs2 = new BitSet(8);
5-         // set() - setting bits
6-         bs1.set(0); bs1.set(1); bs1.set(3); bs2.set(1);
7-         bs2.set(2); bs2.set(3);
8-         System.out.println("BS1: " + bs1);
9-         System.out.println("BS2: " + bs2);
10        // get() - check a bit
11        System.out.println("Bit 0: " + bs1.get(0));
12        // and() - logical AND
13        bs1.and(bs2); System.out.println("AND: " + bs1);
14        // cardinality()
15        System.out.println("Set bits: " + bs1.cardinality());
16    }
17 }
```

Output

```
BS1:  {0, 1, 3}
BS2:  {1, 2, 3}
Bit 0:  true
AND:  {1, 3}
Set bits:  2
```

Explanation

set() marks bit as true

get(0) returns true

and() keeps common bits

cardinality() counts set bits

Date Class – Definition

Definition

The **Date** class represents a **moment in time** with millisecond precision. It stores time as milliseconds since **Jan 1, 1970** (Unix Epoch).

Syntax

```
Date d1 = new Date();  
Date d2 = new Date(long ms);
```

Important Methods

toString()	date as String
getTime()	milliseconds (long)
before(Date d)	true if before d
after(Date d)	true if after d
compareTo(Date d)	compare two dates
equals(Date d)	check equality

DateDemo.java

```
1 import java.util.Date;
2 public class DateDemo{
3     public static void main(String[] a){
4         // Current date
5         Date d1 = new Date(); System.out.println("Now: " + d1);
6         // Milliseconds
7         long ms = d1.getTime(); System.out.println("Ms: " + ms);
8         // Epoch date
9         Date d2 = new Date(0); System.out.println("Epoch: " + d2);
10        // after() and before()
11        System.out.println("After: " + d1.after(d2));
12        System.out.println("Before: " + d1.before(d2));
13        // compareTo()
14        System.out.println("Compare: " + d1.compareTo(d2));
15    }
16 }
```

Output

```
Now: Sun Apr 05 04:55:24 GMT 2026
Ms: 1775364924554
Epoch: Thu Jan 01 00:00:00 GMT 1970
After: true
Before: false
Compare: 1
```

Explanation

new Date() captures current time
getTime() returns milliseconds
new Date(0) is Unix Epoch
after() returns true
before() returns true
compareTo returns int value

Calendar Class – Definition

Definition

Calendar is an **abstract class** providing methods for calendar fields: YEAR, MONTH, DAY, HOUR, MINUTE, SECOND. Use `Calendar.getInstance()`.

Syntax

```
Calendar cal = Calendar.getInstance();
```

Important Methods

<code>get(Calendar.YEAR)</code>	get year
<code>get(Calendar.MONTH)</code>	get month (0-11)
<code>get(Calendar.DATE)</code>	get day of month
<code>get(HOUR OF DAY)</code>	get hour (24-hr)
<code>set(field, value)</code>	set a field
<code>add(field, amount)</code>	add to a field
<code>getTime()</code>	return Date object

CalDemo.java

```
import java.util.*;
public class CalDemo{
    public static void main(String[] a){
        Calendar cal = Calendar.getInstance();
        // get() - individual fields
        int yr = cal.get(Calendar.YEAR); int mo = cal.get(Calendar.MONTH)+1;
        int dy = cal.get(Calendar.DATE); int hr = cal.get(Calendar.HOUR);
        int mn = cal.get(Calendar .MINUTE);
        System.out.println("Year: " + yr); System.out.println("Month: " + mo);
        System.out.println("Day: " + dy);
        System.out.println("Time: " + hr + ":" + mn);
        // add() - add 10 days
        cal.add(Calendar.DATE, 10);
        System.out.println("After 10: " + cal.getTime());
        // set() - specific date
        cal.set(2025, 0, 1); System.out.println("Set: " + cal.getTime());
    }
}
```

Output

Year: 2026
Month: 4
Day: 5
Time: 5:45
After 10: Wed Apr 15 05:45:39 GMT 2026
Set: Wed Jan 01 05:45:39 GMT 2025

Explanation

getInstance() returns current
MONTH is 0-based, add 1
add(DATE,10) moves by 10 days
set(2025,0,1) = Jan 1, 2025

Random Class – Definition

Definition

The **Random** class generates **pseudo-random numbers** of type int, double, float, long, and boolean. A **seed** determines the sequence generated.

Syntax

```
Random r = new Random();  
Random r = new Random(long seed);
```

Important Methods

nextInt()	random int (any range)
nextInt(n)	random int: 0 to n-1
nextDouble()	random double: 0.0 to 1.0
nextFloat()	random float: 0.0 to 1.0
nextBoolean()	random true/false
nextGaussian()	Gaussian double
setSeed(long)	set seed value

RandomDemo.java

```
1 ~ import java.util.Random;
2 ~ public class RandomDemo{
3 ~     public static void main(String[] a){
4         Random r = new Random();
5         // nextInt() - any integer
6         System.out.println("Int: " + r.nextInt());
7         // nextInt(n) - 0 to 99
8         System.out.println("0-99: " + r.nextInt(100));
9         // nextDouble()
10        System.out.println("Double: " + r.nextDouble());
11        // nextBoolean()
12        System.out.println("Bool: " + r.nextBoolean());
13        // nextFloat()
14        System.out.println("Float: " + r.nextFloat());
15        // Dice (1 to 6)
16        int dice = r.nextInt(6) + 1;
17        System.out.println("Dice: " + dice);
18    }
19 }
```

Output

```
Int:  -1234567
0-99:  42
Double: 0.734
Bool:  true
Float:  0.512
Dice:   4
```

Explanation

nextInt() – any integer

nextInt(100) – 0 to 99

nextInt(6)+1 – dice 1 to 6

Values change each run

Formatter – Definition

Definition

Formatter provides **formatted output** similar to C's `printf()`. Uses format specifiers to control output display.

Syntax

```
Formatter fmt = new Formatter();  
fmt.format("format_string", args...);
```

Spec	Type
%d	Integer
%f	Float/Double
%s	String
%c	Character
%b	Boolean
%n	New Line
%.2f	2 decimals
%10d	Width 10

Important Methods

`format()` – formats output
`toString()` – returns String
`close()` – closes Formatter

FmtDemo.java

```
1 import java.util.Formatter;
2 public class FmtDemo{
3     public static void main(String[] a){
4         Formatter fmt = new Formatter();
5         fmt.format("Name: Alice\n");
6         fmt.format("Age: 25\n");
7         fmt.format("Marks: 87.5678\n");
8         fmt.format("ID: 1001\n");
9         fmt.format("Active: true\n");
10        System.out.println(fmt); fmt.close();
11    }
12 }
```

Output

```
Name:  Alice
Age:   25
Marks: 87.57
ID:    1001
Active: true
```

Explanation

%s replaces with string

%.2f rounds to 2 decimals

%10d right-aligns in 10 chars

close() frees resources

Scanner – Definition

Definition

Scanner reads and parses input from keyboard (`System.in`), files, and strings using delimiter-based parsing.

Syntax

```
Scanner sc = new Scanner(System.in);  
Scanner sc = new Scanner(new File("f.txt"));  
Scanner sc = new Scanner("Hello 100");
```

Important Methods

<code>nextInt()</code>	reads int
<code>nextDouble()</code>	reads double
<code>next()</code>	reads one word
<code>nextLine()</code>	reads full line
<code>nextBoolean()</code>	reads boolean
<code>hasNext()</code>	true if more tokens
<code>hasNextInt()</code>	true if next is int
<code>close()</code>	closes Scanner

ScnDemo.java

```
import java.util.Scanner;
public class ScnDemo{
    public static void main(String[] a){
        Scanner sc = new Scanner(System.in);
        // nextLine() - full name
        System.out.print("Name: "); String name = sc.nextLine();
        // nextInt() - age
        System.out.print("Age: "); int age = sc.nextInt();
        // nextDouble() - marks
        System.out.print("Marks: "); double marks = sc.nextDouble();
        System.out.println("--- Info ---");
        System.out.println("Name : " + name);
        System.out.println("Age : " + age);
        System.out.println("Marks: " + marks);
        sc.close();
    }
}
```

Input

```
Alice
25
87.5
```

Output

```
--- Info ---
Name : Alice
Age : 25
Marks: 87.5
```

Explanation

nextLine() reads with spaces

nextInt() reads integer

nextDouble() reads decimal

close() frees resources

Collections – Overview

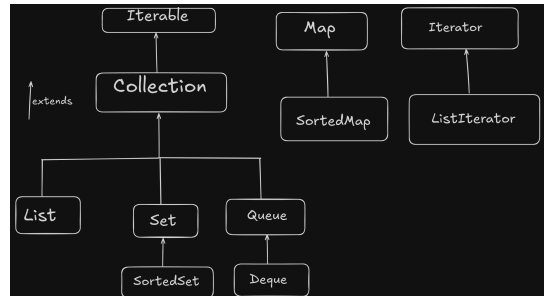
Definition

A **Collection** is a **framework** providing ready-made data structures to **store, manipulate, and retrieve** groups of objects dynamically.

Array vs Collection

Array	Collection
Fixed size	Dynamic size
Same type only	Stores Objects
No methods	Rich methods

H
i
e
r
a
r
c
h
y



Note

Note: Map is **separate**, not under Collection interface.

Collection Interfaces

Interface	Key Characteristics
Collection	Root interface: add, remove, size
List	Ordered, allows duplicates, indexed access. Impl: ArrayList, LinkedList, Vector
Set	No duplicates, unordered. Impl: HashSet, TreeSet
Queue	FIFO. Impl: LinkedList, PriorityQueue
Deque	Double-ended queue. Impl: ArrayDeque
Map	Key-Value pairs, no duplicate keys. Impl: HashMap, TreeMap

Important Methods

<code>add(E e)</code>	<code>remove(Object o)</code>
<code>contains(Object o)</code>	<code>size()</code>
<code>isEmpty()</code>	<code>clear()</code>
<code>iterator()</code>	<code>toArray()</code>

ArrayList – Definition

Definition

ArrayList is a **resizable array** of the `List` interface. Grows dynamically. Fast random access in $O(1)$ time.

Features

- Ordered (insertion order)
- Allows duplicates and null
- Fast random access $O(1)$
- Slow insert/delete middle $O(n)$
- Not thread-safe

Important Methods

<code>add(E e)</code>	add at end
<code>add(i, E e)</code>	add at index
<code>get(i)</code>	get at index
<code>set(i, E e)</code>	replace at index
<code>remove(i)</code>	remove at index
<code>indexOf(e)</code>	find index
<code>size()</code>	element count
<code>contains(e)</code>	check element

ALDemo.java

```
import java.util.*;
public class ALDemo{
    public static void main(String[] a){
        ArrayList<String> list = new ArrayList<>();
        // add() - adding elements
        list.add("Mango"); list.add("Apple");
        list.add("Banana"); list.add("Mango"); // duplicate OK
        list.add(1, "Grapes");
        // index 1
        System.out.println(list); System.out.println("Size: " + list.size());
        // get() - by index
        System.out.println("Idx 0: " + list.get(0));
        // set() - update
        list.set(0, "Cherry"); System.out.println(list);
        // remove()
        list.remove("Banana"); System.out.println(list);
        // contains() indexOf()
        System.out.println("Contains Apple: " + list.contains("Apple"));
        System.out.println("Index: " + list.indexOf("Apple"));
    }
}
```

Output

```
[Mango, Grapes, Apple, Banana, Mango]
Size: 5
Index 0: Mango
[Cherry, Grapes, Apple, Banana, Mango]
[Cherry, Grapes, Apple, Mango]
Contains Apple: true
Index: 2
```

Explanation

Duplicate allowed

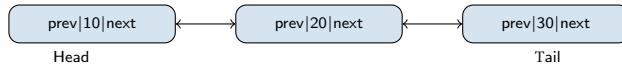
add(1,"Grapes") inserts at 1

set(0,"Cherry") replaces at 0

indexOf returns first index

Definition

LinkedList implements both `List` and `Deque`. Each node has data + next + prev reference (Doubly Linked List).



Features

- Fast insert/delete at ends $O(1)$
- Can act as Stack, Queue, Deque
- Slow random access $O(n)$
- More memory (stores pointers)

Important Methods

<code>addFirst()</code>	add at start
<code>addLast()</code>	add at end
<code>removeFirst()</code>	remove first
<code>removeLast()</code>	remove last
<code>getFirst()</code>	get first
<code>getLast()</code>	get last
<code>push()</code>	stack add
<code>pop()</code>	stack remove

HashSet and TreeSet

HashSet

Uses **hashing**. Unordered output.

No duplicates

One null allowed

$O(1)$ add/search

TreeSet

Uses **Red-Black Tree**. Sorted output.

No duplicates

No null allowed

$O(\log n)$ add/search

SetDemo.java

```
import java.util.TreeSet;
import java.util.HashSet;
class SetDemo{
    public static void main(String...args){
        HashSet<Integer> hs = new HashSet<>();
        hs.add(30); hs.add(10); hs.add(20);
        System.out.println("HashSet: " + hs); // [10, 20, 30]

        TreeSet<Integer> ts = new TreeSet<>();
        ts.add(30); ts.add(10); ts.add(20);
        System.out.println("TreeSet: " + ts); // [10, 20, 30]
    }
}
```

Output

HashSet: [20, 10, 30]
TreeSet: [10, 20, 30]

HashMap

Definition

HashMap stores **Key-Value** pairs. Each key is **unique**, values can repeat. Uses **hashing** for $O(1)$ retrieval.

Important Methods

put(K,V)	insert pair
get(K)	get value
remove(K)	remove pair
containsKey(K)	key exists?
containsValue(V)	value exists?
keySet()	all keys
values()	all values
entrySet()	all pairs
size()	count

MapDemo.java

```
import java.util.*;
class MapDemo{
    public static void main(String...args){
        Map<String, Integer> m = new HashMap<>();
        m.put("Alice", 90); m.put("Bob", 70); m.put("Darth", 50);
        System.out.println("get() method of map: " + m.get("Alice")); // 90
        for(String k : m.keySet())
            System.out.println(k+" : " + m.get(k));
    }
}
```

Output

```
get() method of map: 90
Bob:70
Darth:50
Alice:90
```

Sorting in Collections

Definition

Sorting arranges elements in order. `Collections.sort()` sorts Lists. `Arrays.sort()` sorts arrays.

Syntax

```
Collections.sort(list);  
Collections.sort(list, Collections.reverseOrder());  
Collections.sort(list, comparatorObj);
```

Output

```
Asc: [10, 20, 30, 50]  
Desc: [50, 30, 20, 10]  
Max: 50  
Min: 10
```

SortDemo.java

```
1 import java.util.*;  
2 class SortDemo{  
3     public static void main(String...args){  
4         List<Integer> list = new ArrayList<>();  
5         list.add(10);list.add(20);list.add(30);list.add(50);  
6         Collections.sort(list);  
7         System.out.println("Asc: " + list); // [10,20,30,50]  
8         Collections.sort(list, Collections.reverseOrder());  
9         System.out.println("Desc: " + list); // [50,30,20,10]  
10        System.out.println("Max: " + Collections.max(list));  
11        System.out.println("Min: " + Collections.min(list));  
12    }  
13 }
```

Comparable – Definition

Definition

Comparable (in `java.lang`) defines **natural ordering** of a class. One sorting sequence. Compared inside the class.

Syntax

```
class X implements Comparable<X> {  
    public int compareTo(X obj) {  
        // negative --> this < obj  
        // zero --> this == obj  
        // positive --> this > obj  
    }  
}  
Collections.sort(list);
```

Note

ONE sort sequence (natural ordering)
`compareTo()` must be overridden
Defined INSIDE the class

CompDemo.java

```
import java.util.*;
public class CompDemo{
    public static void main(String[] a){
        ArrayList<Student> list = new ArrayList<>();
        list.add(new Student("Alice", 85)); list.add(new Student("Bob", 72));
        list.add(new Student("Charlie", 90)); Collections.sort(list);
        System.out.println(list);
    }
}
class Student implements Comparable<Student>{
    String name; int marks;
    Student(String name, int marks){this.name = name; this.marks = marks;}
    @Override
    public int compareTo(Student s) { return this.marks - s.marks; }
    public String toString() { return name + ":" + marks; }
}
```

Output

[Bob:72, Alice:85, Charlie:90]

Explanation

Student implements Comparable
sorts by marks ascending
this.marks - s.marks
Collections.sort() calls
compareTo() internally
Output: 72, 85, 90

Comparator – Definition

Definition

Comparator (in `java.util`) allows **multiple custom sorting sequences** externally. More flexible than `Comparable`.

Syntax

```
class MyComp implements Comparator<Type> {  
    public int compare(Type o1, Type o2) {  
        // negative --> o1 < o2  
        // zero --> equal  
        // positive --> o1 > o2  
    }  
}  
Collections.sort(list, new MyComp());
```

Note

- MULTIPLE sort sequences possible
- Defined OUTSIDE the class
- `compare()` method overridden

Comp2Demo.java

```
class Employee{
    String name; int salary;
    Employee(String n, int s){ name = n; salary = s; }
    public String toString(){ return name + ":" + salary; } }
    // Comparator 1: by Name
class ByName implements Comparator<Employee>{
    public int compare(Employee a, Employee b){return a.name.compareTo( b.name); } }
    // Comparator 2: by Salary
class BySalary implements Comparator<Employee>{
    public int compare(Employee a, Employee b) { return a.salary - b.salary; } }

public class Comp2Demo{
    public static void main(String...a){
        ArrayList<Employee> lst = new ArrayList<>();
        lst.add(new Employee("Charlie",50)); lst.add(new Employee("Alice",70));
        lst.add(new Employee("Bob",60));
        Collections.sort(lst, new ByName()); System.out.println("Name: "+lst);
        Collections.sort(lst, new BySalary()); System.out.println("Salary:"+lst);
    }
}
```

Output

Name: [Alice:70, Bob:60, Charlie:50]
Salary:[Charlie:50, Bob:60, Alice:70]

Explanation

Two Comparator classes
ByName sorts alphabetically
BySalary sorts ascending
Same list, two orders
Employee not modified

Comparable vs Comparator

Feature	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo()	compare()
Sequences	ONE	MULTIPLE
Defined	Inside class	Outside class
Flexibility	Less	More
Usage	sort(list)	sort(list, comp)

Use Comparable

ONE default sort order
e.g. Sort Students by Roll No.

Use Comparator

MULTIPLE sort orders
e.g. Sort by Name OR Salary

AllCollections.java

```
import java.util.*;

public class AllCollections {
    public static void main(String[] args) {
        // 1. ArrayList - ordered, allows duplicates
        ArrayList<String> al = new ArrayList<>();
        al.add("Java"); al.add("Python"); al.add("C++");
        Collections.sort(al);
        System.out.println("Sorted: " + al); // [C++, Java, Python]
        // 2. HashSet - no duplicates, unordered
        HashSet<Integer> hs = new HashSet<>();
        hs.add(5); hs.add(3); hs.add(5); // duplicate ignored
        hs.add(1);
        System.out.println("HashSet: " + hs); // order not guaranteed
        // 3. TreeSet - sorted set
        TreeSet<Integer> ts = new TreeSet<>(hs);
        System.out.println("TreeSet: " + ts); // [1, 3, 5]
        // 4. HashMap - key-value pairs
        HashMap<String, Integer> hm = new HashMap<>();
        hm.put("Alice", 90); hm.put("Bob", 75); hm.put("Charlie", 85);
        System.out.println("HashMap:");
        for (Map.Entry<String, Integer> e : hm.entrySet()) {
            System.out.println(e.getKey() + ": " + e.getValue());
        }

        // 5. Queue using LinkedList (FIFO)
        Queue<String> queue = new LinkedList<>();
        queue.add("A"); queue.add("B"); queue.add("C");
        System.out.println("Queue: " + queue);
        System.out.println("Removed: " + queue.poll()); // removes A
        System.out.println("After poll: " + queue);
        // 6. PriorityQueue - elements sorted automatically
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        pq.add(30); pq.add(10); pq.add(20);
        System.out.println("PriorityQueue: " + pq);
        System.out.println("Removed (min): " + pq.poll()); // smallest element
        System.out.println("After poll: " + pq);
        // 7. Collections utility methods
        System.out.println("Max: " + Collections.max(al));
        System.out.println("Min: " + Collections.min(al));
        Collections.reverse(al);
        System.out.println("Reversed: " + al);
    }
}
```

Unit III – Summary

Packages

package keyword
import keyword
CLASSPATH variable
private ; default
protected ; public

Utility Classes

StringTokenizer
BitSet
Date and Calendar
Random
Formatter
Scanner

Collections

List – ordered, dup
Set – no duplicates
Queue – FIFO
Map – key-value
Comparable
Comparator

Scenario	Use
Ordered and Duplicates	ArrayList
No Duplicates	HashSet
No Duplicates and Sorted	TreeSet
Key-Value	HashMap
One sort order	Comparable
Multiple sort orders	Comparator

UNIT III COMPLETE

Key Packages

<code>java.lang</code>	String, Math, System
<code>java.util</code>	Collections, Date, Scanner
<code>java.io</code>	File I/O operations